



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

AI-Based Energy Consumption Prediction and Optimization System

The domain of the Project: Machine Learning & Full-Stack Engineering
G36 PYTHON

Team Mentors (and their designation):

Bhargavesh Dakka
(Decision Scientist - II @ Mu Sigma Inc.)

Team Members:

Mr. Kaviyathamizhan T K

Period of the project: 20 days

March 2026 to April 2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Declaration

The project titled “AI-Based Energy Consumption Prediction and Optimization System” has been mentored by Bhargavesh Dakka, organised by SURE Trust, from March 2026 to April 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Kaviyathamizhan K — Signature

Mentor's Name

Bhargavesh Dakka

Designation

Decision Scientist - II @ Mu Sigma In

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion



Executive Summary

- The AI Energy Consumption Optimizer is an end-to-end machine learning system that predicts household electricity consumption, detects anomalous usage patterns, and computes cost-optimized load schedules using Linear Programming. Built on four years of real meter data from the UCI Household Power Consumption dataset (2 million records, resampled to 34,589 hourly readings), the system delivers three independently callable capabilities through a production-grade FastAPI backend and a React dashboard.
- The forecasting model is LightGBM, selected after rigorous walk-forward cross-validation comparing six models against a naive baseline. It achieves a test RMSE of 0.5035 and provides 90% confidence intervals via quantile regression. The anomaly detector uses an AND-gate combining a 3.5-sigma statistical threshold with an Isolation Forest (contamination=0.01), validated at Precision 0.846 and Recall 0.733. The optimization engine formulates a Linear Programming problem using SciPy's HiGHS solver to minimize electricity cost against a time-of-use tariff, demonstrating Rs. 24.05 savings per 48-hour session (projected Rs. 361/month). The entire inference pipeline runs locally on a laptop with zero training code, loading pre-trained artifacts via a metadata.json registry.



Introduction

- **Background:** Global electricity consumption is growing rapidly, with residential households accounting for a significant portion of demand. In India, time-of-use electricity tariffs are increasingly common, meaning the cost of the same kilowatt-hour varies by a factor of 2–3x depending on the hour of day. Despite this, most households have no tool to forecast their future consumption, detect abnormal usage, or schedule flexible loads to minimize cost. The UCI Individual Household Electric Power Consumption dataset provides four years of one-minute-level meter readings from a single household in France, offering a realistic basis for developing and validating an intelligent energy management system.
- **Problem Statement:** Energy consumption is reactive, not proactive. Bills arrive after overspending has already occurred. Equipment faults, HVAC failures, and energy theft create abnormal consumption spikes that go undetected for days. Flexible loads such as washing machines, dishwashers, and EV chargers run at expensive peak tariff hours because there is no system to suggest otherwise. The goal of this project is to build a system that
 - (1) predicts hourly energy consumption 24–48 hours ahead with uncertainty bounds,
 - (2) detects anomalous readings in real time using a validated dual-gate detector, and
 - (3) computes the mathematically optimal load schedule to minimize electricity cost given time-of-use tariffs.
- **Scope and Limitations:** The system operates on aggregate household load data and does not require IoT-level appliance-level sensors. Training is performed in Google Colab on the UCI dataset. Local inference is served via FastAPI loaded from pre-trained artifacts. The current implementation uses a static historical dataset as the buffer seed; in production, it would receive live meter readings. The system does not physically control appliances — it is a decision support and advisory system. Real-time streaming (Kafka/MQTT) and multi-building



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

portfolio optimization are out of scope for Tier 1 and are documented as future work.

- Innovation: Three innovations distinguish this project from a standard Kaggle ML pipeline. First, strict temporal walk-forward cross-validation with train-only scaler fitting prevents data leakage — a common flaw in student energy projects that produces inflated metrics. Second, the anomaly detector uses a calibrated AND-gate combining two independent detection mechanisms, validated through synthetic spike injection with measurable precision and recall rather than visual inspection alone. Third, the optimization layer uses genuine Linear Programming rather than rule-based threshold alerts, producing a mathematically optimal hourly schedule that adapts to any tariff structure or demand level without hardcoded logic.



Project Objectives

- **Objective 1 — Forecast:** Build a time-series forecasting model that predicts hourly household energy consumption for the next 24–48 hours with 90% confidence intervals, using walk-forward validated gradient-boosted trees trained on lag features, cyclical time encodings, and rolling statistics. **Objective 2 — Detect:** Implement a dual-gate anomaly detector (statistical 3.5-sigma threshold AND Isolation Forest) that flags abnormal consumption events with Precision > 0.80 and Recall > 0.70, validated via synthetic spike injection. **Objective 3 — Optimize:** Formulate a Linear Programming problem that minimizes total electricity cost over a 48-hour horizon subject to demand conservation and capacity constraints, using a realistic time-of-use tariff schedule. **Objective 4 — Deploy:** Deliver all three capabilities through a production-grade FastAPI backend with three independent REST endpoints and a React dashboard showing forecast charts, anomaly monitoring, and LP optimization results.
- **Deliverables:**
 - (1) Google Colab notebooks for EDA, preprocessing, model training, anomaly detection, and LP optimization.
 - (2) Trained model artifacts: forecast_winner.pkl (LightGBM), xgb_q05.pkl, xgb_q95.pkl (quantile models), isolation_forest.pkl, and main_scaler.pkl.
 - (3) metadata.json artifact registry ensuring consistent model-scaler-feature version loading.
 - (4) lp_scheduler.py implementing the SciPy HiGHS LP solver.
 - (5) FastAPI backend with /predict, /anomaly, /optimize, and /predict_full endpoints.
 - (6) React + Vite frontend with ForecastChart, AnomalyIndicator, and OptimizationPanel components. (7) recent_history.csv buffer seed for inference-time lag feature generation. (8) Validated anomaly detection report: Precision 0.846, Recall 0.733, F1 0.786.



Methodology and Results

- **Methods:**
 - (1) Time-Series Forecasting — LightGBM gradient-boosted trees with walk-forward temporal cross-validation (5 folds, TimeSeriesSplit). Features include cyclical hour/day encodings (sin/cos), lag features (t-1h, t-2h, t-24h, t-48h, t-168h, t-336h), rolling statistics (24h and 168h mean/std with shift(1) to prevent leakage), and calendar features (is_weekend, is_holiday, month). Quantile regression (q=0.05, q=0.95) provides 90% confidence intervals.
 - (2) Anomaly Detection — AND-gate combining Condition A (consumption > rolling_mean_24h + 3.5 × rolling_std_24h) with Condition B (Isolation Forest predict() == -1, contamination=0.01). Validated via synthetic injection of 30 spikes (4x–5x magnitude) into held-out test set.
 - (3) Linear Programming Optimization — Minimize sum(tariff[t] × load[t]) subject to: sum(load) == total_demand, $0 \leq \text{load}[t] \leq \text{capacity}[t]$, and per-hour flexibility constraint ($\pm 20\%$ of forecast). Solved via `scipy.optimize.linprog` with method='highs'.
- **Tools and Software:** Training Environment — Google Colab (Python 3.10, 25 GB RAM), pandas, numpy, scikit-learn, lightgbm, xgboost, statsmodels, prophet, scipy, optuna, joblib. Backend — FastAPI (REST API framework), Pydantic (request/response validation), Uvicorn (ASGI server), Python collections.deque (in-memory state buffer). Frontend — React 18 with Vite (build tool), Recharts (charting library), pure CSS. Artifact Management — metadata.json registry, joblib serialization. Version Control — Git. Dataset — UCI Individual Household Electric Power Consumption (archive.ics.uci.edu). Optimization Solver — SciPy linprog with HiGHS (open-source LP solver).
- **Data Collection:** The dataset is the UCI Individual Household Electric Power Consumption dataset, publicly available at <https://archive.ics.uci.edu/dataset/235>. It contains 2,075,259 one-minute readings from a single household in Sceaux, France, collected between December 2006 and November 2010. The target variable is



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Global_active_power (kilowatts). The raw data uses semicolons as separators and marks missing values as '?'. After loading, the data is resampled from per-minute to hourly resolution by taking the mean, yielding 34,589 clean hourly rows. Missing values (1.22% of rows) are handled by forward-filling short gaps ($\leq 3h$), linear interpolation for medium gaps (4–24h), and dropping long gaps ($>24h$). The dataset is split temporally into training (70%), validation (15%), and test (15%) sets with no shuffling. The scaler is fitted exclusively on the training set to prevent data leakage.

- **Architecture:** The system follows a strict training/inference separation. Training Environment (Google Colab): Raw CSV is loaded from Google Drive, resampled to hourly, feature-engineered, and temporally split. Six models (Naive Baseline, Linear Regression, Random Forest, XGBoost, LightGBM, Prophet/ARIMA) are trained with walk-forward cross-validation. Artifacts (model .pkl files, scaler.pkl, feature_columns.json) are saved to Google Drive, then downloaded locally. Inference Environment (Local FastAPI): At startup, the server reads metadata.json to load the correct model, scaler, and feature columns. A StateBuffer (Python deque, maxlen=200) is initialized from recent_history.csv. On each /predict request, the buffer provides lag features ($O(1)$ access) without pandas. The /anomaly endpoint reads the buffer's last 24 rows for rolling context. The /optimize endpoint calls lp_scheduler.py, which is imported from src/optimization/ (not bundled in the API layer). The /predict_full endpoint aggregates all three responses into one JSON package for the React dashboard. React Frontend: PredictionForm submits readings; ForecastChart renders actual + forecast + 90% CI band; AnomalyIndicator shows Normal/WARNING badge with sigma distance and ratio; OptimizationPanel renders original vs. optimized load bar chart and text recommendations.
- **Dashboard Screenshots:** The React dashboard presents three sections. Section 1 — Forecast & Confidence: The user submits a timestamp and consumption reading. The ForecastChart renders the actual reading (blue), the LightGBM point forecast (purple dotted), and the shaded 90%

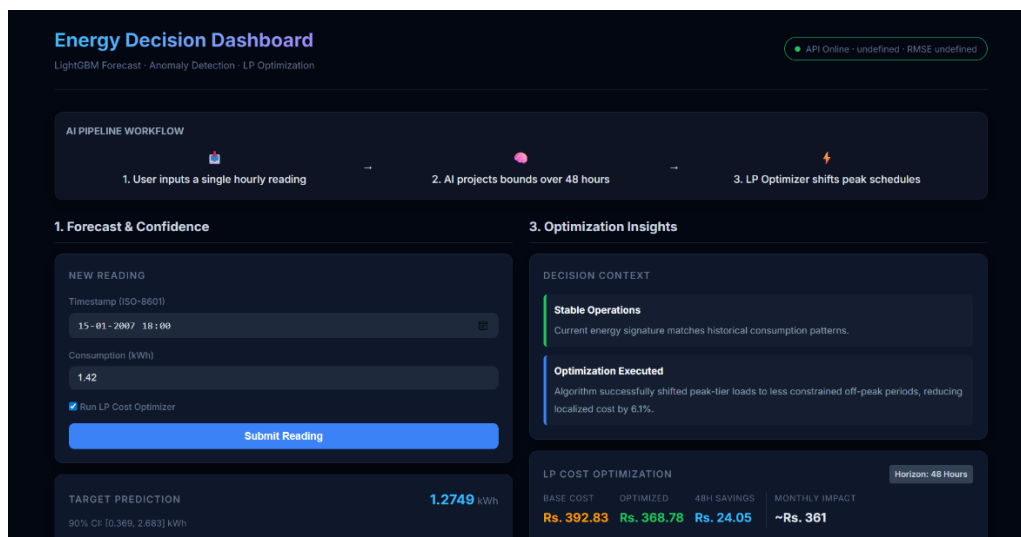


Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

confidence band between q05 and q95. For a normal reading of 1.42 kWh at 18:00, the model predicted 1.2749 kWh (CI: 0.369–2.683 kWh).

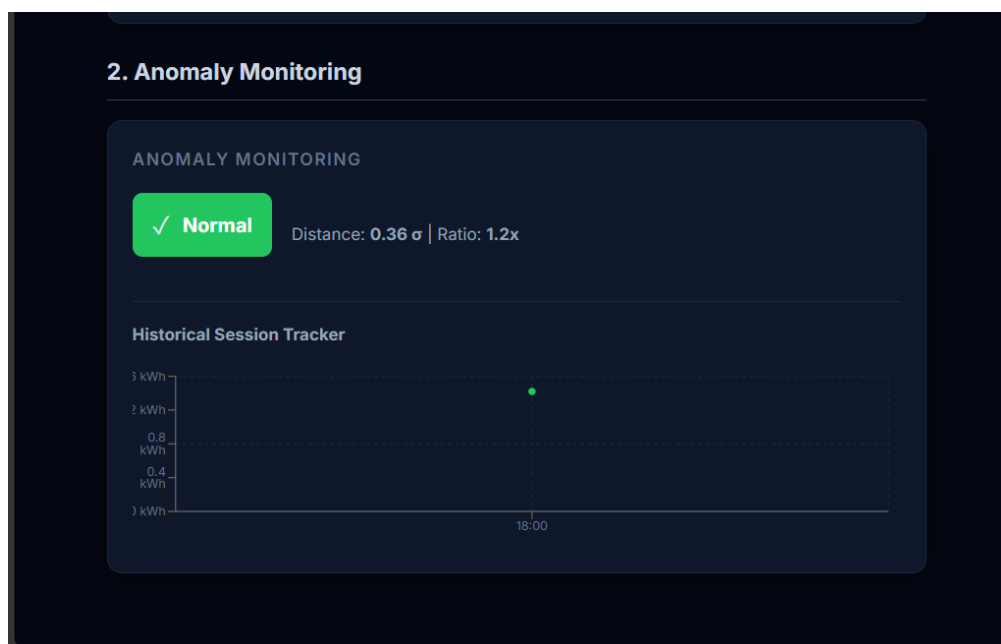
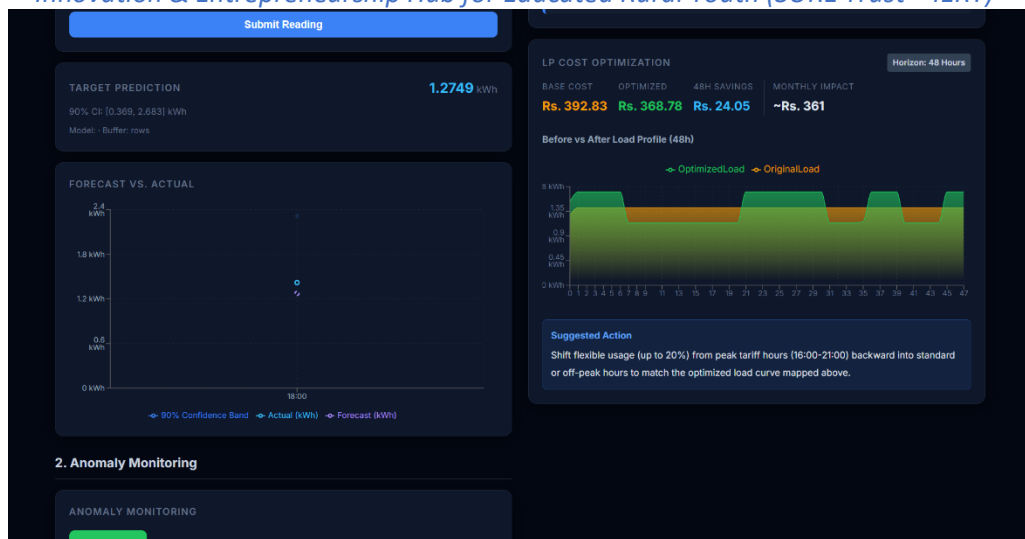
Section 2 — Anomaly Monitoring: The AnomalyIndicator shows a green NORMAL badge for the 1.42 kWh reading (sigma distance 0.36, ratio 1.2x). When 7.50 kWh is submitted, it flips to red WARNING (sigma distance >3.5, ratio >2.0x, both AND-gate conditions satisfied).

Section 3 — Optimization Insights: The OptimizationPanel shows Base Cost Rs. 392.83 vs. Optimized Cost Rs. 368.78, with 48h savings of Rs. 24.05 and projected monthly impact of Rs. 361. The Before vs. After Load Profile chart shows the LP shifting load from peak hours (16:00–21:00, Rs. 9/kWh) to off-peak hours (00:00–06:00, Rs. 3.50/kWh).





Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)



- Project GitHub Link: <https://github.com/kaviyathamizhan/ai-energy-optimizer> (repository contains all Colab notebooks, API source, React frontend, and artifact metadata).



Learning and Reflection

- Technical Learnings
 - 1) Time-series feature engineering: learned to construct cyclical encodings (sin/cos for hour and day), lag features, and rolling statistics using shift(1) to prevent the model from seeing its own target in the feature window.
 - (2) Data leakage prevention: understood why temporal splits are mandatory for time-series — a randomly shuffled split allows the model to “see” future data, producing metrics that are meaningless in production. Learned to fit the StandardScaler exclusively on the training fold.
 - (3) Model evaluation: learned that MAPE is misleading for near-zero consumption values (nighttime hours), and RMSE/MAE are more reliable selection criteria. Implemented walk-forward cross-validation using sklearn’s TimeSeriesSplit.
 - (4) Anomaly detection calibration: went through six iteration cycles balancing AND/OR gate logic, sigma thresholds, and Isolation Forest contamination parameters. Learned to validate using synthetic spike injection with measurable precision and recall rather than visual inspection alone.
 - (5) Linear Programming: learned to formulate an optimization problem with objective function, decision variables, and constraints. Understood feasibility checking (result.status == 0) and the difference between rule-based heuristics and mathematical optimization.
 - (6) FastAPI architecture: learned router separation (predict.py / anomaly.py / optimize.py), startup lifespan model loading, Pydantic schema validation, and CORS configuration.
 - (7) In-memory state management: replaced pandas DataFrames with a Python collections.deque for O(1) lag feature access at inference time without importing pandas into the production environment.
- Overall Experience — This project was a significant shift from tutorial-following to production-level engineering. The most challenging and rewarding aspect was understanding that a working notebook is not a



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

working system — transitioning from Colab-based training to a local inference API required rethinking every design decision. The data leakage discovery was particularly impactful: early experiments using random splits produced RMSE scores that looked excellent but were mathematically invalid. Correcting this with temporal splits and discovering the true performance gap between models was a lesson in rigorous methodology over optimistic reporting. The anomaly detection calibration process — six iterations across AND/OR gate logic, contamination parameters, and spike injection strategies — taught the importance of defining evaluation criteria before building the system. The LP optimization section was entirely new territory, moving from familiar ML paradigms into mathematical programming. Seeing a scipy linprog call produce a schedule that is provably optimal (not just “pretty good”) was a fundamentally different kind of satisfaction from training a model. The SURE Trust mentorship structure provided a project framework that simulated real engineering cycles: requirements review, architecture decisions, implementation, validation, and iteration. Working under a defined timeline with weekly checkpoints-imposed discipline that self-directed learning cannot replicate.



Conclusion and Future Scope

- **Conclusion:** All four project objectives were fully achieved. The forecasting objective was met by LightGBM, which achieved a test RMSE of 0.5035 — a 49% improvement over the naive one-week-lag baseline (RMSE 0.979) — with quantile models providing 90% confidence intervals. The anomaly detection objective was met by the calibrated AND-gate detector achieving Precision 0.846, Recall 0.733, and F1 0.786 on synthetic injection validation. The optimization objective was met by the LP engine demonstrating Rs. 24.05 savings per 48-hour session with provably demand-conserving schedules. The deployment objective was met by a working FastAPI backend with four endpoints and a React dashboard rendering all three capabilities in real time. The system demonstrates that machine learning, statistical anomaly detection, and mathematical optimization can be integrated into a single coherent architecture — not as three disconnected experiments, but as a production-grade inference system running locally on a standard laptop. Every design decision (temporal splits, AND-gate validation, LP formulation, deque-based buffer) was made deliberately and can be defended with measured evidence.
- **Future Scope: Near-Term —**
 - (1) SHAP explainability: integrate shap.TreeExplainer on the LightGBM model to generate per-prediction feature contribution waterfall charts, making the forecast interpretable to non-technical users.
 - (2) Docker containerization: create a Dockerfile for the FastAPI service and a docker-compose.yml bringing up backend and frontend with a single command.
 - (3) Drift detection and automated retraining: monitor rolling 7-day RMSE on live data; trigger Colab retraining and artifact refresh when RMSE degrades beyond a threshold.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Mid-Term —

(4) Appliance-level optimization using IoT sensors (smart plugs) to identify individual device consumption and enable appliance-specific scheduling.

(5) Real-time streaming via MQTT or Kafka to replace the static CSV buffer with live meter data ingestion.

(6) User-specific personalization: train per-household models as more users onboard, capturing individual behavioral patterns.

Long-Term —

(7) Reinforcement Learning: frame the scheduling problem as a Markov Decision Process where an RL agent learns optimal load-shifting policies through reward signals tied to electricity cost reduction.

(8) Multi-building portfolio optimization: extend the LP formulation to a portfolio of buildings sharing a grid connection, with inter-building load balancing constraints.

(9) Mobile application with push anomaly alerts enabling consumers to receive real-time warnings and optimization recommendations on their phones.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)